pyaces Improvement Plan

Review of structure, build and tests — reference commit `3a123417` (2026-10-23)

> **Scope**: this review covers the layout of `pyaces/`, its build (`build.sh`), the f2py interface and the test tooling. It combines the known issues in chapter 11 of the developer manual (`docs/pyaces_developer_manual.pdf`) with items found on a second pass. Current state: pyaces 51/51 PASS and an identical 296-entry link audit on both the local workstation and the ccsd server.

Summary

#	Item	Area	Priority	Size
1	Per-module partial links that reproduce the classic name resolution	Correctness	High	Large (needs a prototype)
2	An ACES error terminates the Python interpreter	Robustness	High	Medium–large
3	Check for calls bound to COMMON blocks	Correctness	High	Small
4	Fail the build when a rename does not apply	Correctness	High	Small
5	Remaining known defects (Getlstpy, Get1ehpy, Runpccd)	Correctness	Medium	Small–medium
6	Stop f2py from generating the module twice	Build	Medium	Medium
7	Remove the hard-coded Python version	Build	High	Small
8	Incremental builds	Build	Low	Large
9	Compiler and flag cleanup (mpicc, -qopenmp)	Build	Low	Small
10	Relax build-environment assumptions (module, ifort check)	Build	Medium	Small
11	Remove legacy files (about 47 MB)	Structure	Medium	Small
12	Remove duplicated headers	Structure	Medium	Small
13	Move the rename list into a data file	Structure	Medium	Medium
14	A Python package and API layer	Structure	Medium	Medium–large
15	Fix `Python/moio_len.py`	Structure	Low	Small
16	Gaps in the test roll-up and the classifier	Tests	High	Small
17	API unit tests	Tests	Medium	Medium

A. Correctness and robustness

1. Per-module partial links that reproduce the classic name resolution High

Problem

- A single `.so` holding about 107 archives depends on about 60 hand-written rename pairs (`EXTRA_LOCALIZE_SYMBOLS`).
- A new module or a different toolchain can introduce new collisions, and the audit baseline already holds 296 known differences.

Proposal

- Partially link each entry module (vcc, vee, lambda, ...) with `ld -r` together with its own link list (`ACES_LINK_LISTS`), so each module gets the same routines as its classic executable.
- Then hide every symbol except the entry point with `objcopy`.
- The rename list should become unnecessary and the audit should drop to near zero.

Caveat: COMMON blocks that modules share in-process (`/flags/`, `/icore/`, ...) must stay global, and the `.so` will grow. Prototype with one module first.

2. An ACES error terminates the Python interpreter High

- `aces_exit` calls `c_exit`, which ends the whole process, so Python cannot catch the error.
- Given the one-calculation-per-process rule as well, add an API layer that runs each calculation in a subprocess. The test runner (`pyaces_suite_runner.py`) already works this way.

3. Check for calls bound to COMMON blocks High

- The `hbar_` defect fixed on 2026-10-23 (Runhbar calling lambda's `COMMON /HBAR/`) is invisible to the current audit, because there is no `T hbar_` anywhere.
- Add a check that fails the build when a symbol a pyaces `.o` calls (`U`) resolves to data (`B`/`D`) in the `.so`.

4. Fail the build when a rename does not apply High

- Today a typo, or a pair for a module that is not processed, is silently ignored (`2>/dev/null || true`).
- Check that the symbol exists in that archive and that the module is processed; otherwise fail the build.

5. Remaining known defects Medium

- `Getlstpy` ignores its `Irrep` and `Ndim` arguments and uses `Left_index` as the irrep. Clean up the arguments or make them match their documented meaning.
- Verify the symmetry-block offset in `Get1ehpy` (`get1ehpy.F:101`) on a molecule with symmetry.
- The runner calls `Init()` before `Runpccd`, which calls Joda itself. This path has never been tested.

B. Build and compilation

6. Stop f2py from generating the module twice Medium

- The `.pyf` and the source scan each generate the module, and the second overwrites the first, so the intents and COMMON declarations in the `.pyf` are never applied.
- Pick one source of truth: `Cf2py` directives in the sources, or the `.pyf` alone.
- f2py's `--lower` option could replace the `sed` lower-casing workaround. That would make the Python names lower-case, which changes the API.

7. Remove the hard-coded Python version High

- The output name is fixed to `aces2py.cpython-312-x86_64-linux-gnu.so`, so other Python versions cannot import it.
- Use the output of `python3 -c "import sysconfig; print(sysconfig.get_config_var('EXT_SUFFIX'))"`.

8. Incremental builds Low

- Every build deletes `gen/` and `build/` entirely and unpacks about 37 archives again.
- Splitting the steps with make or meson would make rebuilds after small changes much faster.

9. Compiler and flag cleanup Low

- The C wrapper is compiled with `mpicc`, but nothing uses MPI. Plain `icx` or `gcc` is enough.
- The Fortran shims are built with `-qopenmp`, which may be unnecessary with sequential MKL. Check, then clean up.

10. Relax build-environment assumptions Medium

- `pyaces/build.sh` always runs `module purge/load`, so it fails in setvars/path environments or where there is no `module` command. Make it follow configure's `ACES_PROGRAM_ENV_MODE`.
- The top-level `build.sh` checks for `ifort` before the pyaces `ifort` shim is on PATH, so it refuses an ifx-only toolchain.

C. Structure

11. Remove legacy files Medium

- Unused files (about 47 MB in total): `Fxx/` (45 MB, about 6,100 files), `include/`, `a2_pyf/include_no/`, the `Archive/` folders, `Exp/`, `rename.py`.
- Also unused: 35 stale `.f` files and `suite_classification.tsv.before_*`.
- How: keep them on a separate tag or branch and delete them from master; the history stays intact.

12. Remove duplicated headers Medium

- The nine `Main/*.com` files are old copies that differ from aces2's headers (explicit `integer*8`, no include guards).
- Use the aces2 headers instead, so the two cannot drift apart.

13. Move the rename list into a data file Medium

- About 60 pairs and their explanations are buried in the comments of a 510-line script.
- Move them to a data file with one pair and its reason per line (e.g. `tools/localize.txt`), so the build and the audit read the same file.

14. A Python package and API layer Medium

- Today users call the raw f2py functions: mixed-case names, results passed back through caller-supplied arrays, record names padded to 8 characters.
- Add a thin `pyaces` package with lower-case functions, return values, exceptions and a `pyproject.toml`. Design it together with item 2 (subprocess execution).

15. Fix `Python/moio_len.py` Low

- It writes dictionaries as `"key" = value`, which is a syntax error, and it has duplicate keys.
- Turned into a valid dictionary module, it can serve as the table of list numbers.

D. Tests

16. Gaps in the test roll-up and the classifier High

- The pyaces roll-up in `run_tests.sh` does not count MISMATCH or TIMEOUT, so failures can drop out of the summary.
- Rerunning `classify_suite.py` would erase the hand-edited TSV rows (KS → Runintgrt, 117a/b/c → Rundirmp2, excluded cases). Move those exceptions into the code.
- The runner's default run directory is a HiPerGator path (`/blue/...`).

17. API unit tests Medium

- Only energies are tested today.
- Add small tests for record read/write, `Getreclenpy` / `Cgetrecpy`, each `Runhbar` branch and list access.

Recommended order

Stage	Items	Why
1	3, 4, 7, 16	Low risk, a few hours of work. Closes the paths through which defects pass silently.
2	11, 12, 15	Structural cleanup. No change in behaviour; makes later work easier.
3	6, 13, 14	Usability and maintainability. API names may change, so this needs agreement with users.
4	1, 2, 8	Large structural changes. Decide after a prototype shows the benefit and the cost.

After every change, confirm aces2 125/125, pyaces 51/51 and matching link-audit results on both the local workstation and ccsd.